

Real-Time Bangla / Banglish Live-Mic STT for a Medical Pre-Screening Capstone: The Practical Build Plan

TL;DR

- Start this week with the browser Web Speech API (Chrome, `lang="bn-BD"`) for instant free live Bangla dictation, then build the robust path on faster-whisper (CTranslate2, int8 CPU) with a `small`/Bangla-fine-tuned model streamed over a FastAPI WebSocket — this two-track plan gives you a working demo immediately and a defensible, offline-capable, hardware-appropriate system for the thesis.
- On your no-NVIDIA AMD hardware, run STT CPU-only with int8: tiny/base Whisper run faster than real-time and are good for live streaming; `small` is near real-time; Bangla accuracy is the hard constraint (real-world WER ~25-45% out-of-the-box), so treat raw transcription as imperfect and put a separate free-LLM correction stage (Gemini Flash / Groq / OpenRouter) AFTER it — never let correction touch the stored raw transcript.
- Architecture: browser mic (MediaRecorder/AudioWorklet) → WebSocket → FastAPI (async) → faster-whisper, with a swappable correction module and a transliteration module. Keep the raw utterance immutable, do normalization/Banglish→Bangla in a later stage, and structure the backend as a clean API so a mobile app can reuse it later.

Key Findings

1. No usable free *true-streaming* offline Bangla STT exists today. Vosk — the obvious lightweight streaming engine — has effectively no maintained Bangla model (its download links are dead; an open GitHub issue requests re-upload with no resolution). So real-time Bangla on your hardware means either (a) the browser Web Speech API (Google's cloud engine, free, online-only) or (b) chunked "near-real-time" Whisper on CPU. There is no free Kaldi-style true-streaming Bangla option.
2. Whisper is the accuracy leader for Bangla, but only when Bangla-fine-tuned. Vanilla `whisper-large-v3` is poor on real Bangla (a 2026 benchmark, arXiv 2603.19256, reports 75.0% WER on competition audio zero-shot). The community standard is `tugstugi/whisper-medium` (BengaliAI fine-tune), with raw baselines around 34% WER (34.07% on the Bengali-Loop benchmark) and fine-tuned systems reaching 16-27% WER on long-form Bangla. For medical Bangla specifically, the fine-tuned Whisper BanglaASR achieved 9.05% WER (vs 17.25% for DeepSpeech2) on a 5.8-hour held-out test set, per Rahman et al., "Automatic Speech Recognition for Biomedical Data in Bengali Language" (arXiv 2406.12931) — trained on a 57.59-hour Bengali medical corpus mapping 1,264 unique English medical symptoms to Bengali and Sylheti dialects. `(arxiv + 5)`
3. Your AMD hardware dictates CPU-only int8 inference. CTranslate2 (faster-whisper's engine) has no ROCm backend — it is CUDA or CPU only. whisper.cpp can use the AMD GPU via Vulkan, but on the RX 570 (old GCN) and integrated Vega this is unreliable (silent CPU fallback is a documented, common failure, e.g. Buzz issue #1443). Plan for CPU; treat any AMD-GPU speedup as a bonus, not a dependency. The whisper-

ctranslate2 docs confirm the CPU sweet spot: "On CPU int8 will give the best performance." (PyPI)

4. **CPU performance is fine for small models.** With int8: tiny ~10× real-time, base ~6× real-time, small ~2-7× real-time on a modern x86 CPU; medium is around/below real-time and not recommended for streaming on a 6-core CPU. The SYSTRAN faster-whisper README measured (small) int8 at ~7.6× real-time (1m42s to transcribe 13 min, 1,477 MB RAM) on an i7-12700K at 8 threads; expect a 6-core Ryzen to be modestly slower. RAM footprint: tiny ~0.27 GB, base ~0.39 GB, small ~0.85 GB (faster-whisper int8 ~1.5 GB), medium ~2.1 GB, large-v3 ~3.9 GB. Both your machines (24 GB and 12 GB RAM) comfortably fit up to medium.
5. **Free LLM correction is viable and should be swappable.** Gemini 2.5 Flash/Flash-Lite are capped at **1,500 requests/day with 1M TPM (15 RPM Flash, 30 RPM Flash-Lite)** on the free tier (Google's official rate-limits page, updated 2026-05-28; note Gemini 2.5 Pro's free tier was removed in April 2026, and RPD resets at midnight Pacific). Groq free tier is **30 RPM / 6,000 TPM / 1,000 RPD** for most models (and Whisper STT free at 2,000 audio requests/day), applied at the organization level. OpenRouter offers many (:free) models at ~20 RPM (50/day, rising to 1,000/day after a one-time \$10 credit purchase). All are OpenAI-compatible, so you can hot-swap providers behind one interface. (TokenMix + 4)
6. **Faithful-capture-first is an architecture rule, not a model choice.** Capture and persist the raw ASR output immutably; do Banglish→Bangla transliteration and normalization in separate, later stages (IndicXlit, mBART banglish-to-bangla models, or the LLM). This directly satisfies your non-negotiable "never alter the patient's exact words."

Details

1. Real-time live STT options compared (Module 1)

Web Speech API (browser, Chrome) — the quick-start winner. Chrome's (SpeechRecognition) supports (bn-BD) live-mic transcription for free, streaming interim + final results, using Google's cloud engine. Pros: zero setup, genuinely live "as you speak," decent Bangla accuracy (same engine as Google's products), works on Windows + Linux (Chrome/Chromium). Cons: **requires Chrome + internet; does not give you the raw audio bytes** (so you can't re-process or archive the waveform); not available in all browsers; not offline; you're at the mercy of Google's model. For a clinic pre-screening tool the no-raw-audio limitation matters long-term, but it is unbeatable for a Week-1 demo and for collecting real Banglish test utterances.

faster-whisper (CTranslate2) — the robust recommendation. Up to 4× faster than openai/whisper at equal accuracy — per the SYSTRAN faster-whisper README: "This implementation is up to 4 times faster than openai/whisper for the same accuracy while using less memory. The efficiency can be further improved with 8-bit quantization on both CPU and GPU" (benchmarks executed with 8 threads on an Intel Core i7-12700K). int8 on CPU is the sweet spot for your machines. CPU-only (no ROCm), runs identically on Windows + Linux, pip-installable, bundles FFmpeg via PyAV (no system ffmpeg needed). Use Silero VAD (built in) to cut silence and reduce hallucination. Convert any Bangla-fine-tuned HF Whisper checkpoint to CTranslate2 with (ct2-transformers-converter --model <hf_model> --output_dir <dir> --

`quantization int8`). This is the best balance of accuracy, cross-platform stability, and integration for a 4-month build. `(GitHub + 4)`

whisper.cpp — the AMD-GPU-curious alternative. Pure C/C++, GGML/GGUF, runs on CPU and can use Vulkan/OpenCL for AMD (and ROCm/HIP, but only for newer GCN/RDNA arch — the RX 570 is too old to be a first-class ROCm target). A Vulkan build hit ~7.5-8× real-time on a modern RX 9070 XT in community tests, but on old GCN (RX 570) and Vega iGPU the Vulkan path is flaky (documented silent CPU fallback). Good as a CPU fallback or to experiment with GPU offload, but don't make it the backbone. `(Maroonmed)`

OpenAI Whisper (original), WhisperX, whisper-streaming. Original Whisper is slow on CPU — don't use directly. WhisperX adds alignment/diarization (useful later for doctor/patient separation) but is batch-oriented, and wav2vec2 alignment on Bangla performed poorly in studies (one reported WER 0.79 with the WhisperX + Bengali alignment combo).

`(whisper_streaming)` (UFAL) is the proven real-time wrapper: per Macháček, Dabre & Bojar, "Turning Whisper into Real-Time Transcription System" (arXiv 2307.14743), "*Whisper-Streaming achieves high quality and 3.3 seconds latency on unsegmented long-form speech transcription test set*" — specifically 3.3 s average on the English ESIC European Parliament test set running on an NVIDIA A40 GPU, using the LocalAgreement-2 policy, with faster-whisper as the recommended backend. Adopt its streaming policy rather than rolling your own (but expect higher latency on CPU; the paper notes CPU performance tests were pending).

`(arxiv + 8)`

Vosk (Kaldi). Truly streaming, ultra-light (50 MB models), zero-latency streaming API — BUT no maintained Bangla model (download links dead). Effectively unusable for Bangla today. Listed for completeness; do not build on it.

Wav2Vec2 Bangla (arijitx, tanmoyio, ai4bharat IndicWav2Vec). CTC models, decent for files, but not designed for low-latency streaming and generally beaten by fine-tuned Whisper on Bangla (IndicWav2Vec Bengali >40% WER zero-shot in the 2603.19256 benchmark).

`(arijitx/wav2vec2-large-xlsr-bengali)` and the `(xls-r-300m-bengali)` variant (trained on OpenSLR SLR53) are the best-known; use as comparison baselines, not the primary engine.

`(arxiv)` `(Hugging Face)`

Bangla-specific projects. `(BanglaSpeech2Text)` (pip `(banglaspeech2text)`, wraps shhossain/whisper-*-bn models, includes a mic helper via SpeechRecognition) is the easiest Bangla Whisper on-ramp but its small models are weak (whisper-base-bn ~46% WER).

`(tugstugi/whisper-medium)` (BengaliAI) is the strongest community backbone.

IndicConformer/IndicWhisper (ai4bharat) are alternatives. Crucially for your project, fine-tuning on clinical code-switched Bangla works dramatically well: Hossain et al., "MediBeng Whisper Tiny" (medRxiv 2025.04.25.25326406), a Whisper Tiny fine-tuned on the synthetic MediBeng Bengali-English code-switched clinical dataset using just 20% of the data, reports **0.01 Word Error Rate and a 0.98 BLEU score** on mixed-language clinical input — a strong proof that domain fine-tuning is the real accuracy lever and an excellent thesis direction.

`(Hugging Face)` `(medRxiv)`

Free streaming cloud APIs supporting Bangla. Google Cloud Speech-to-Text: **60 free minutes/month perpetual** (no expiry), true gRPC streaming with interim results, bn-BD, Chirp models — good but tiny free quota. Deepgram/AssemblyAI/Soniox/Gladia/ElevenLabs Scribe: trial credits only, variable Bangla support, not free long-term. For your budget, the Web Speech API (free, unlimited, bn-BD) beats all paid streaming APIs as the cloud option. (Google)

2. Bangla / Banglish / dialects + preserving exact words

Architecture rule: two stages, raw is immutable.

- **Stage A (capture):** store the exact ASR output verbatim in a `raw_transcript` field that is never mutated. This is your evidentiary record of "what the patient said."
- **Stage B (normalize):** produce a separate `normalized_transcript` (and optionally `roman_transcript`) via transliteration/LLM. Display the corrected version but always keep raw retrievable.

Banglish / Roman Bangla handling. Code-switching raises WER 30–50% and is genuinely hard; Bangla-English LID + ASR research (IEEE 10796602) reports ~21.5% WER with 70% LID accuracy, and the MUCS-2021 Bengali-English code-switch set had 22.9–27.3% out-of-vocabulary rates. For Roman→Bangla script: **AI4Bharat IndicXlit** (`ai4bharat-transliteration`), MIT, supports `bn`, both directions, beam search, trained on the 26M-pair Aksharantar dataset) is the standard; note its dependency on fairseq can be painful to install on Windows (use WSL/Linux or pin versions). mBART/seq2seq community models (`Mdkaif2782/banglish-to-bangla`, `shadabtanjeed/mbart-banglish-to-bengali-transliteration`, `kazalbrur/...`) are pip-friendly via transformers but weaker on code-mixed input (their own model cards warn they don't handle code-switching well). For Bangla text normalization, use the csebuetnlp BanglaBERT normalizer pipeline. Practically, the free LLM (Gemini/Groq) is often the most robust single tool for "clean this Banglish into proper Bangla while preserving meaning," because it handles code-switching where dedicated transliteration models fail.

(nih + 4)

3. Downstream text correction via free AI API

Recommended free options (2026), swappable behind one interface:

- **Google Gemini Flash (AI Studio free tier):** best first choice — 1,500 RPD, 1M TPM, 15 RPM (Flash) / 30 RPM (Flash-Lite), 1M-token context, no card, handles Bangla well. Caveat: Google may use free-tier prompts for training (privacy concern for medical data), and the 2.5 Pro free tier was removed in April 2026. (TokenMix) (Find Skill.ai)
- **Groq:** fastest (LPU), OpenAI-compatible, 30 RPM / 1,000 RPD / 12K–100K TPD on most models; great for short correction calls. (Groq also offers free Whisper STT at 2,000 requests/day — a possible cloud STT fallback.) (TokenMix) (Eesel AI)
- **OpenRouter:** widest model variety via `:free` models, ~20 RPM, 50/day (or 1,000/day after a one-time \$10 credit purchase); single OpenAI-compatible key with failover.
- **Cerebras** (high daily token volume, ~1M tokens/day) and **Mistral** (free Experiment tier, requires training opt-in) as backups.

Design: wrap correction in a `Corrector` interface with a provider enum; read API keys from env; implement retry + provider fallback (Gemini → Groq → OpenRouter) because every free tier has hard caps. Send a strict prompt: *"Correct spelling/grammar of this Bangla/Banglish medical utterance; do NOT add, remove, or infer symptoms; return only corrected text."* Keep raw separate so a bad correction never corrupts the record. **Privacy caveat:** free tiers may log/train on data — for real patient data later you'd need a paid no-training tier or a local LLM (Ollama); fine for a prototype with synthetic/consented data.

4. Cross-platform real-time audio architecture

Recommended stack:

- **Browser capture:** `getUserMedia` → `AudioContext({sampleRate:16000})` + **AudioWorklet** (preferred over the deprecated `ScriptProcessor`) to grab 16-kHz mono PCM; or `MediaRecorder` (webm/opus) decoded server-side via FFmpeg. `AudioWorklet` + raw PCM gives lowest latency and avoids server-side decode. (lifix) (GitHub)
- **Transport: WebSocket** sending **binary** frames in 20–40 ms packets for transport (buffered to ~0.5–1 s windows for Whisper). Use `wss://` in production.
- **Backend: FastAPI + native WebSockets** (async, ideal for streaming) over `Flask-SocketIO` — FastAPI's async model handles concurrent audio streams cleanly and is the community default for Whisper streaming servers (`WhisperLiveKit`, `whisper_streaming_web`).
- **Audio libs:** `sounddevice` (`PortAudio`; auto-installs the DLL on Windows via pip, needs `libportaudio2`/`portaudio` via pacman on Arch) for server-side capture/testing; `webrtcvad` or Silero VAD for voice-activity detection; faster-whisper bundles FFmpeg so you avoid a system ffmpeg dependency for decoding. (Python Sounddevice)
- **Chunking + partial results:** buffer ~0.5–1 s, run VAD, transcribe a rolling window, display "unvalidated" interim text in grey then commit finalized text (the `whisper_streaming_web` pattern). Final-emission latency $\approx 2 \times$ chunk size (≈ 2 s with 1 s chunks). (GitHub)
- **Mobile-friendliness:** keep STT/correction behind a clean WebSocket + REST API with JSON contracts; a future React Native/Flutter app reuses the same backend by opening the same WebSocket. Don't couple audio logic to the web UI.

Windows vs Linux gotchas: On Arch install `portaudio` and `ffmpeg` via pacman; on Windows pip wheels bundle `PortAudio` (note: the pip Windows DLL defaults to the non-ASIO build — fine for you). Pin everything in `requirements.txt` and use a venv (or conda for thornier native deps like `fairseq`/`IndicXlit`, since conda may override the pip-bundled `PortAudio`). Keep one `requirements.txt` that works on both; isolate the painful `fairseq`-based transliteration into an optional extra so it doesn't block the core install. (DeepWiki) (DeepWiki)

5. Recommended concrete stack + phased plan

Recommended starter stack (this hardware):

- Python 3.10/3.11, venv, single `requirements.txt`.
- Backend: **FastAPI** + **uvicorn**, native WebSockets.
- STT: **faster-whisper** (CTranslate2), `compute_type="int8"`, model = start with `small` (or `base` for live feel); upgrade to a CTranslate2-converted `tugstugi/whisper-medium` for accuracy when latency budget allows. Silero VAD on.
- Quick-start STT: **browser Web Speech API** (`lang="bn-BD"`).
- Correction: **swappable Corrector** → Gemini Flash (primary), Groq + OpenRouter (fallback).
- Transliteration/normalization: **IndicXlit** (`ai4bharat-transliteration`) and/or LLM; BanglaBERT normalizer.
- Audio: AudioWorklet (browser) + web rtcvad/Silero; `sounddevice` for tests.
- Frontend: plain HTML/JS first (adapt `whisper_streaming_web's live_transcription.html`), React later.

Phased plan:

- **Phase 0 / Week 1 (instant feedback):** Web Speech API page (`bn-BD`) → display raw + send to Gemini for correction → show corrected. Proves the whole raw→correct→display loop with zero ML setup, and lets you collect real Banglish utterances.
- **Phase 1 (robust core, weeks 2–5):** FastAPI WebSocket + faster-whisper (`small`/`base` int8) streaming with VAD and interim results; persist immutable raw + corrected. Cross-platform install verified on both machines.
- **Phase 2 (Bangla accuracy, weeks 5–9):** swap in CTranslate2 `tugstugi/whisper-medium`; add IndicXlit transliteration + LLM normalization; build the swappable correction with fallback; evaluate WER on your own clinic-style Bangla/Banglish samples.
- **Phase 3 (stretch / thesis contribution):** fine-tune Whisper on medical Bangla (à la MediBeng / BanglaASR) or collect a small medical corpus; optional WhisperX for speaker separation; mobile API hardening.

Realistic accuracy expectations. Expect real-world Bangla WER roughly 25–45% out-of-the-box (worse with dialect, code-switching, noise; better with fine-tuning toward ~15–20% in-domain, and as low as ~9% — or near-zero on synthetic clinical sets — only with medical/code-switch fine-tuning). Hardest challenges, in order: (1) code-switched Banglish, (2) regional dialects (Sylheti/Chittagonian differ sharply from standard Bangla), (3) medical terminology and drug names (out-of-vocabulary), (4) noisy clinic audio. This is exactly why the system must "narrow the search space for doctors, not diagnose," keep raw words verbatim, and prioritize emergency detection downstream.

6. "Vibe coding" with Claude Code over a 4-month, 15-module project

Proven patterns from current Claude Code practice (Anthropic docs + community):

- **Keep** `CLAUDE.md` **lean** (<200 lines), **high-signal, project-specific**. Cover WHY/WHAT/HOW: one-line project description, build/test/run commands, conventions,

"always do X" rules (e.g., "NEVER mutate raw_transcript", "system never diagnoses"). Anthropic's own guidance notes frontier models follow ~150-200 instructions reliably and smaller/non-thinking models far fewer — generic filler ("be a senior engineer") measurably does nothing. (Medium)

- **Use progressive disclosure: pointers, not dumps.** Put detailed docs in an `agent_docs/` (or `.claude/rules/`) folder and list them in `CLAUDE.md` with one-line descriptions so Claude reads them only when relevant. Your requested files map cleanly to this:
 - `constitution.md` (non-negotiables: raw words immutable, no diagnosis, emergency-first, free/OSS, AMD/CPU constraints) — referenced from `CLAUDE.md`.
 - `milestone_log.md`, `current_task.md`, `changelog.md`, `test_log.md`, `decisions.md` (ADR-style), `codebase_map.md`, `session_protocol.md`.
- **Session protocol:** start each session by reading `current_task.md` + milestone log; end by updating changelog/decisions/current_task. This is the community-proven "write things down as you learn, read them back at session start" loop. Modern Claude Code also has **auto memory** (shipped v2.1.59, Feb 2026) that records inferred conventions automatically — run `/memory` to see what it already learned and avoid duplicating it. (Medium)
- **Prefer pointers over copies; don't paste code into docs** (it goes stale). Enforce true non-negotiables with hooks/CI (e.g., a `PreToolUse` hook or a test that fails if raw transcript is mutated), not prose — docs are context, not enforcement. (Claude)
- **Prune outdated sections regularly** — a stale `CLAUDE.md` describing an old architecture is worse than none.

Recommendations

1. **Build the Week-1 Web Speech API demo first.** It de-risks the whole pipeline (raw→correct→display), gives your supervisor something live immediately, and harvests real Banglish/dialect test data. Threshold to move on: once you can capture, correct, and store ~50 real utterances.
2. **Make faster-whisper int8 the backbone, default model `small` (or `base` for live feel).** Benchmark on BOTH machines; if streaming latency >3 s or the 5500U laptop struggles, drop to `base`. Upgrade to CTranslate2 `tugstugi/whisper-medium` only if WER is unacceptable AND latency stays usable.
3. **Enforce the immutable-raw rule in code and CI**, not just docs — it's your project's defining ethical/clinical constraint and maps directly to your "never alter the patient's exact words" rule.
4. **Wrap correction + transliteration as swappable modules** with provider fallback (Gemini→Groq→OpenRouter). Benchmark each on your own Banglish samples; switch primary if rate limits bite.
5. **Defer AMD-GPU acceleration.** Try whisper.cpp + Vulkan as an experiment only; if you see silent CPU fallback, abandon it. Don't let GPU chasing eat your 4 months.

6. **Pin a single cross-platform** `requirements.txt` + `venv`; isolate `fairseq/IndicXlit` as an **optional extra**. Verify clean install on Windows and Arch in Phase 1, not at the end.
7. **Set up the Claude Code markdown system on day one** with a lean `CLAUDE.md` pointing to `agent_docs/`, and add a hook/test that guards the raw-transcript invariant.

Benchmarks that change the plan: if live latency on the laptop exceeds ~3 s with `small`, switch to `base` (or keep cloud Web Speech for live and use Whisper for post-hoc accuracy); if Bangla WER on your real samples exceeds ~40% after `tugstugi-medium`, prioritize collecting a small fine-tuning set over adding features.

Caveats

- **Free-tier rate limits and model availability change frequently** (Groq cut daily limits in 2026; Gemini 2.5 Pro's free tier was removed in April 2026; OpenRouter free models can be removed without notice). Verify current quotas before relying on them, and design for failover.
- **Free LLM/cloud STT tiers may log or train on your data** — acceptable for a prototype with synthetic/consented data, NOT for real patient data without a paid no-training tier or a local LLM.
- **CPU performance figures are extrapolated** from the i7-12700K (faster-whisper README), AMD Ryzen AI articles, and whisper.cpp community benchmarks; your exact Ryzen 3500X/5500U numbers will differ — benchmark directly. A 6-core Ryzen will be modestly slower than the cited 8-thread i7.
- **Bangla WER figures vary widely by dataset** (long-form competition audio vs. short clean clips vs. synthetic clinical sets); the 9.05% medical figure (arXiv 2406.12931) and the 0.01 WER MediBeng figure (medRxiv 2025.04.25.25326406) are from fine-tuned studies on their own (partly synthetic) test sets and will NOT generalize to your noisy clinic audio without similar fine-tuning.
- **Web Speech API discards raw audio** and depends on Google's servers — fine for quick-start, but the offline/auditable requirement ultimately favors the local Whisper path.
- **whisper.cpp Vulkan on old AMD GCN/Vega is unreliable** (documented silent CPU fallback); treat any AMD-GPU speedup as unverified on your specific cards until you test it.
- **Vosk Bangla is currently a dead end** — no maintained model; do not plan around it despite its otherwise-ideal streaming properties.